

Lucrarea nr. 2

1. Scopul lucrării

Lucrarea are ca scop însușirea structurii și modului de lucru cu microcontrolerul dsPIC33F. În cadrul lucrării se va prezenta mediul de programare MPLAB IDE v8.21, și se va descrie modul în care se pot realiza aplicații cu dsPIC33F, cu ajutorul acestui program.

2. Prezentarea mediului de programare MPLAB IDE v8.21

Mediul de programare MPLAB IDE (*MPLAB Integrated Development Environment*) reprezintă un program software destinat dezvoltării aplicațiilor cu microcontrolere și microprocesoare Microchip, PIC (*Programmable Interface Controller*[1]) și dsPIC (*Digital Signal PIC*). Programul este denumit mediul de dezvoltare integrat, deoarece acesta oferă mai multe posibilități de a dezvolta coduri pentru microcontrolere.

MPLAB IDE reprezintă o varietate de instrumente hardware și software atât pentru programarea componentelor, cât și pentru configurarea unui întreg sistem.

Un circuit integrat este de obicei un sistem care se folosește de un microcontroler, cum ar fi PIC sau DSC (*Digital Signal Controller*). Aceste microcontrolere conțin atât microprocesor, cât și circuite periferice, constituindu-se în module de control de dimensiuni foarte mici. Aceste circuite pot fi integrate în alte aparate electronice și mecanice pentru dispozitive de control de cost redus.

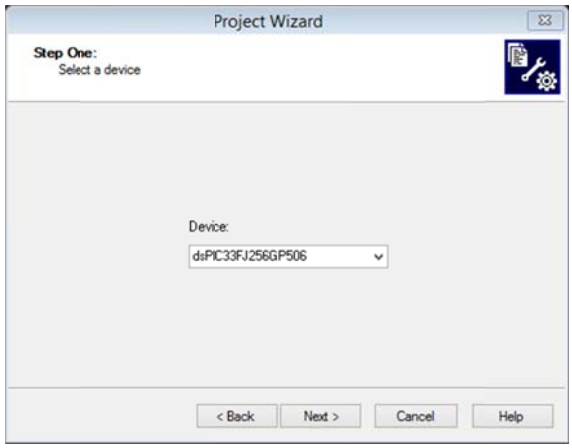
2.1. Instrucțiuni de implementare a unei aplicații cu sistemul de dezvoltare MPLAB IDE

Pentru realizarea unei aplicații în MPLAB IDE trebuie câțiva pași de bază. O modalitate ușoară pentru a realiza acest lucru este folosirea aplicației *Project Wizard*. După deschiderea programului MPLAB IDE se va accesa această aplicație din meniul *Project*. Pașii pentru realizarea unei aplicații sunt descriși în continuare.

1. Selectarea dispozitivului

Se va selecta dispozitivul necesar aplicației dorite. Selecția dispozitivului trebuie însă făcută în etapa de proiectare a aplicației, în funcție de rolul acestuia.

[1] Acronimul *PIC* semnifica inițial *Peripheral Interface Controller*. Ulterior, semnificația s-a schimbat în *Programmable Interface Controller*, găsindu-se chiar și cu sensul *Programmable Intelligent Computer*.



O altă modalitate de a selecta dispozitivul dorit este prin accesarea panoului *Select Device* din meniul *Configure*.



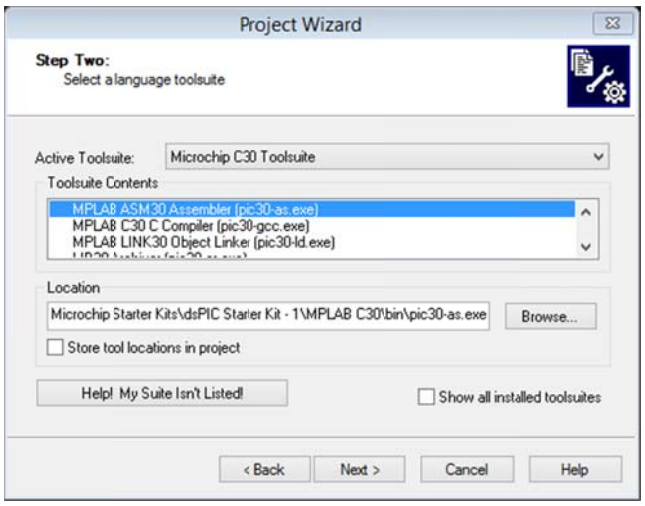
După cum se observă, se poate selecta și familia din care face parte dispozitivul. Culorile LED-urilor indică ce componente MPLAB IDE susțin acest dispozitiv. Semnificația culorilor este următoarea:

- verde – suport deplin;
- galben – suport preliminar pentru o parte viitoare a instrumentului particular al mediului MPLAB;
- roșu – niciun suport pentru dispozitiv. Sprijinul poate urmează a fi implementat sau poate fi nepotrivit pentru instrument.

În ambele cazuri se va selecta *dsPIC33FJ256GP506*.

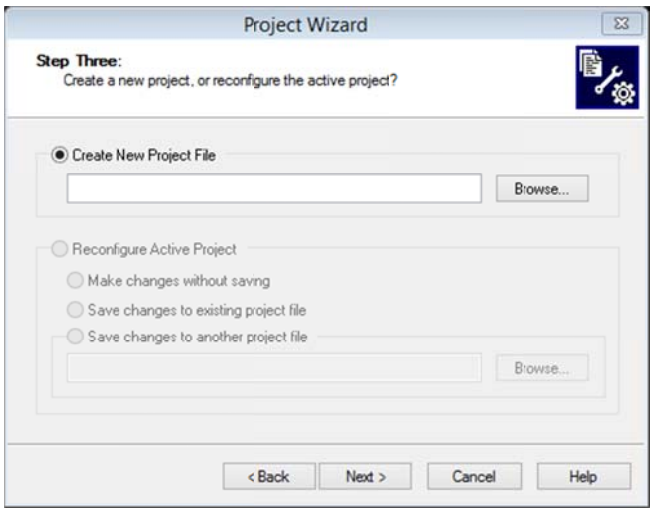
2. Selectarea compilatorului

Compilatorul (*Language Toolsuite*) se alege din meniul aplicației *Project Wizard*. Pentru această aplicație vom folosi bibliotecile și compilatorul C30.



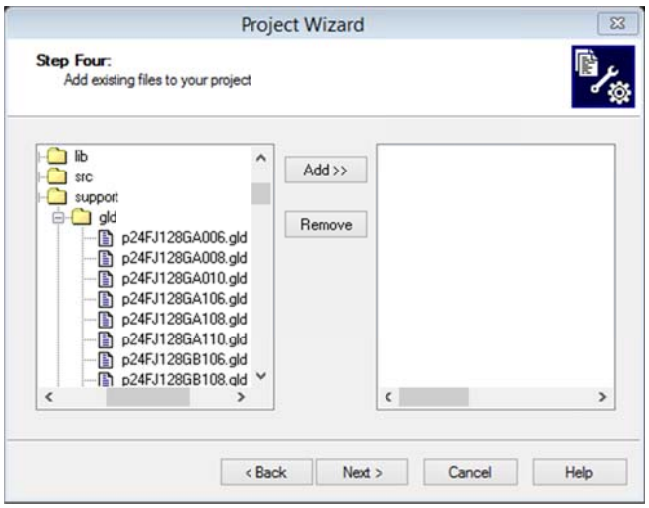
3. Crearea unui proiect

Se va alege locația în care se va salva proiectul: *D:\Projects\Laborator1*. În cazul în care directorul nu există, veți fi întrebat dacă doriți să îl creați. Selectați *OK*.



4. Adăugarea fișierelor în proiect

Se vor adăuga fișierele necesare acestei aplicații. De asemenea, pot fi folosite și fișiere care au fost utilizate în alte aplicații.



Un document sursă nu a fost selectat încă, așa că vom folosi fișiere standard MPLAB IDE. Acestea sunt fișiere care pot fi folosite pentru a începe un proiect. Din locația *C:\Microchip Starter Kits\dsPIC Starter Kit - 1\MPLAB C30\support* se vor adăuga fișierele:

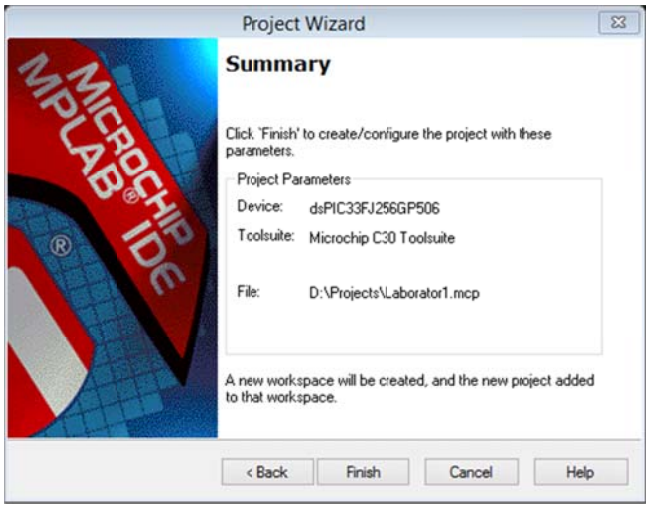
- *p33FJ256GP510.h*, din directorul *h*;

Fișierul antet (*header*) conține definiții pentru toți regiștrii microcontrolerului. Acestea pot fi folosite la scrierea codului.

- *p33FJ256GP506.gld*, din directorul *gld*.

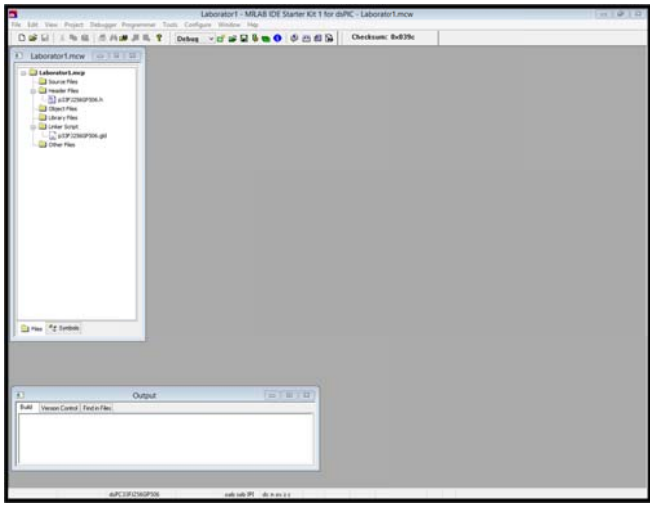
Fișierul *gld* (folosit de aplicația *Object Linker*) conține adresele la care pot fi accesați regiștrii.

Ecranul final este un rezumat ce ilustrează dispozitivul folosit, numele proiectului și bibliotecile utilizate.



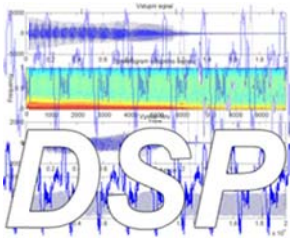
5. Crearea codului

După apăsarea butonului *Finish*, se vizualizează proiectul în ecranul MPLAB IDE. Dacă proiectul nu este deschis se va selecta *View → Project*.



Se va scrie codul programului. Se deschide un fişier nou selectând *File → New*. Se va salva fişierul în directorul *D:\Projects*, ca fişier sursă *.c*, alegând opţiunea *Save As Type – C Source File*. Se va adăuga proiectului fişierul sursă: click dreapta pe *Source Files → Add Files…* şi se alege fişierul salvat anterior. În acest fişier se va scrie codul programului.





6. Compilare proiect

Proiectul va fi compilat, asamblat și trimis într-un cod mașină care poate rula în dispozitivul selectat. Se selectează *Programmer → Select Programmer → 3 Starter Kits*. Se compilează proiectul: click dreapta pe *Laborator1.mcp → Build All.* Dacă nu apar erori, se poate trece la programarea microcontrolerului: *Programmer → Program*.

7. Testarea prin simulare

În final, proiectul va fi testat cu ajutorul simulatorului (*Debugger*). Se va selecta *Debugger → Select tool → 11 Starter Kits*. Se deschide panoul de observare al variabilelor și al regiștrilor: *View → Watch*, după care se alege registrul sau variabila de interes și să selectează *Add Symbol*. Se vor stabili puncte de întrerupere a execuției (*breakpoints*), prin dublu click pe linia de program dorită, și se rula programul. În momentul în care acesta atinge punctul de întrerupere, execuția va fi oprită și se pot vizualiza valorile regiștrilor sau ale variabilelor observate. Se poate continua execuția din acest punct în mod continuu sau pas cu pas.

3. Prezentarea microcontrolerului și a platformei de dezvoltare

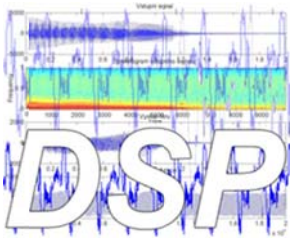
Microcontrolerul dsPIC33FJ256GP506 face parte din familia de microcontrolere dsPIC33. Această familie include circuite cu un număr mare de pini (64, 80 și 100), dimensiune mare a memoriei de program (64*kocteți*, 128*kocteți* și 256*kocteți*), și cu memorie RAM de capacitate mărită (8*kocteți*, 16*kocteți* și 30*kocteți*), iar viteza de procesare este până la 40*MIPS*. Această caracteristică face ca familia de circuite să fie utilizată pentru o mare varietate de aplicații în procesarea digitală a semnalelor.

Această familie de microcontrolere, în general, este compatibilă cu pini familiei de circuite PIC24H, și de asemenea are un mare grad de compatibilitate cu familia de circuite dsPIC30F. Aceasta permite migrarea ușoară între familiile de circuite, ceea ce poate fi util pentru o anumită funcționalitate.

Familia de circuite dsPIC33 utilizează o arhitectură pe 16 biți care integrează caracteristicile de control al unui microcontroler (MCU – *Microcontroller Unit*) cu capacitatea de calcul a unui procesor de semnal digital (DSP – *Digital Signal Processor*). Funcționalitatea rezultată este ideală pentru aplicații care se bazează pe viteză mare de calcul, precum și de control.

Nucleul DSP prezintă acumulatori dubli pe 40 de biți, resurse hardware pentru operațiile de divizare, o gamă largă de regiștri de lucru pe 16 biți și o mare varietate de moduri de adresare a datelor care, împreună cu unitatea centrală de procesare (CPU – *Central Processing Unit*) are și o capacitate extinsă de prelucrare matematică. Ca și microcontroler gestionează un număr considerabil de întreruperi. În plus, prin accesarea directă a memoriei (DMA – *Direct Memory Access*) permite





transferul de date între mai multe periferice fără a mai implica nucleul controlerului.

Spațiul de date poate fi adresat ca 32*k* cuvinte sau 64*kocteți* și este împărțit în două blocuri, menționate ca memoriile de date *X* și *Y*. Fiecare bloc de memorie are propria unitate de generare a adreselor (AGU – *Address Generation Unit*) independentă. Clasa de instrucțiuni a microcontrolerului operează exclusiv prin AGU a memoriei *X*, care accesează harta întreagă a memoriei ca un spațiu liniar de date. Anumite instrucțiuni DSP operează prin unitățile de generare a adreselor *X* și *Y* pentru a suporta operații duble de citire, care împarte spațiul de adresare date în două părți. Spațiul de date *X* și *Y* este limitat de specificațiile dispozitivului.

Memoriile tampon (*buffers*) de adresare circulară sunt suportați în ambele spații de adresare *X* și *Y*. Adresarea circulară a AGU *X* poate fi folosită cu orice clasă de instrucțiuni a microcontrolerului. De asemenea, AGU *X* sprijină adresarea de biți reversivi pentru a simplifica foarte mult intrarea sau ieșirea de date pentru reordonarea secvențelor de date precum la transformata Fourier rapidă (FFT – *Fast Fourier Transform*).

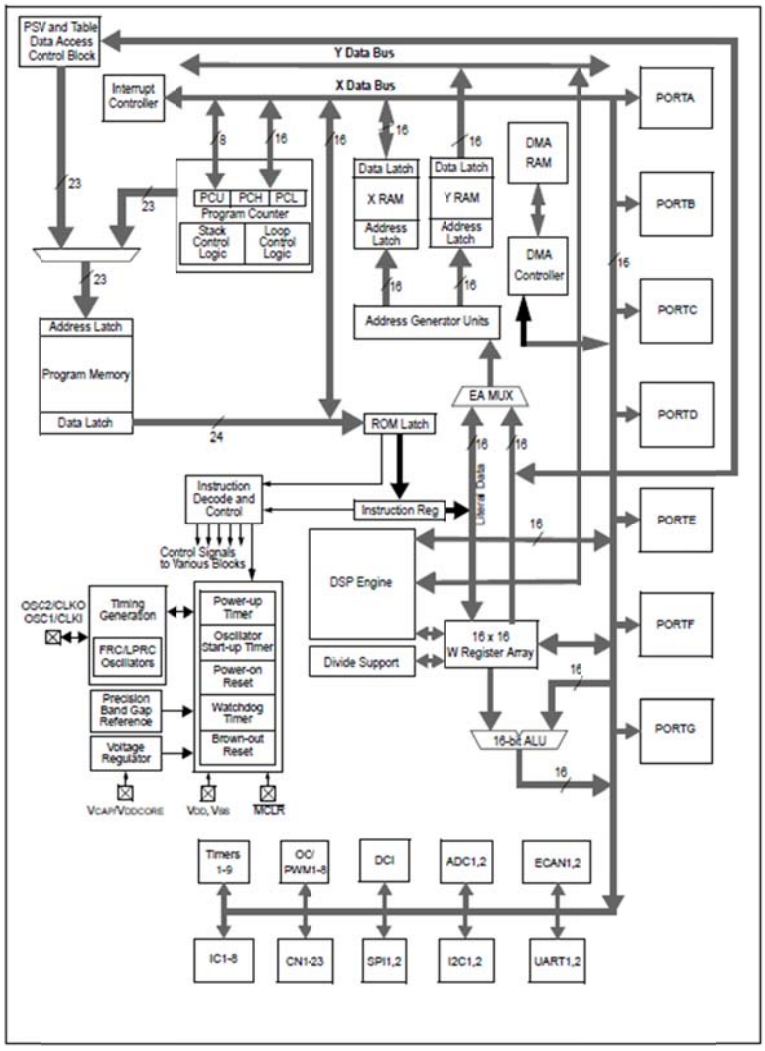
Nucleul DSP este format dintr-un multiplicator de mare viteză, un sumator pe 40 de biți și o unitate de deplasare.

Circuitele dsPIC33FJ256GP506 au arhitectura fluxului de instrucțiuni pe un singur ciclu mașină, prin urmare funcționarea concomitentă a nucleului DSP cu fluxul de instrucțiuni de la MCU nu este posibilă. Totuși, anumite resurse ale ALU (*Arithmetic Logic Unit*), MCU și ale nucleului DSP pot fi folosite concomitent de aceleași instrucțiuni. De asemenea, nucleul DSP are capacitatea de a efectua operații intrinseci între acumulatori fără a necesita date suplimentare.

Câteva din caracteristicile microcontrolerului dsPIC33FJ256GP506 sale sunt următoarele:

- arhitectură Harvard modificată;
- 40 *MIPS* (*Million Instructions Per Second*);
- suportă *DMA* (*Direct Memory Access*);
- memorie *flash* cu capacitate de 256 kB;
- memorie *SRAM* cu capacitate de 30 kB, din care 2kB reprezintă *DMA RAM*;
- module de comunicații *SPI*, I^2C, *UART*;
- convertoare analog-numerice și numeric-analogice.

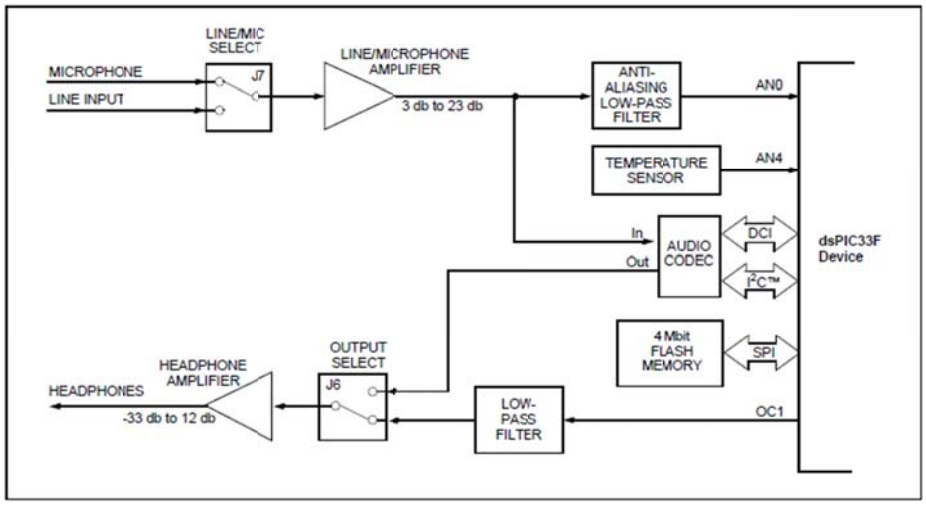
Schema bloc a microcontrolerului:



Acest microcontroler face parte din sistemul *Starter Kit for dsPIC Digital Signal Controllers*, creat de către firma Microchip în special pentru dezvoltarea aplicațiilor de procesare a semnalului vocal. Sistemul se conectează la portul USB al calculatorului, permițând fi programarea şi depanarea folosind programul MPLAB. Semnalele audio sunt preluate cu ajutorul unui microfon extern sau alte echipamente audio, sunt transmise către modulul de conversie analog-numerică, sau către codecul audio pe 24 de biți, WM8510. De asemenea, semnalele de ieşire pot fi generate ca impulsuri modulate în durată (*PWM*), care apoi sunt filtrare trece

jos, sau pot fi furnizate cu ajutorul codecului audio. Sistemul conține și o memorie flash de capacitate de 4*MB*.

Schema bloc a sistemului utilizat:

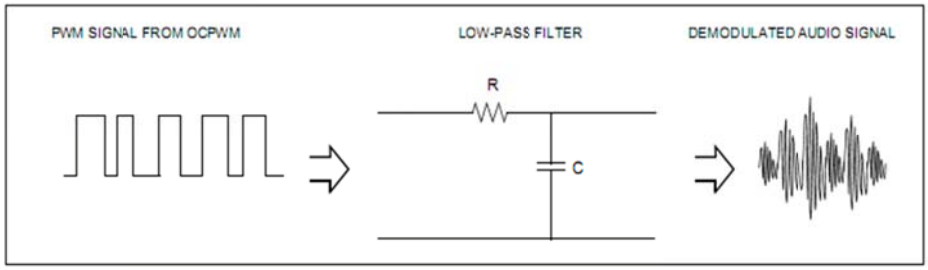


Semnalul audio de la intrare poate veni de la o linie de intrare sau de la un microfon. Selecția se face prin jumperul *J7*. Semnalul selectat este amplificat de un amplificator neinversor (*Line/Microphone Amplifier*) și transmis către intrările de conversie analog digitală ale controlerului dsPIC33F printr-un filtru de tip antialiere, care elimină componentele ce depășesc frecvența care nu mai îndeplinește criteriul lui Nyquist (este un filtru trece jos activ, de tip Sallen Key, de ordinul 6, având frecvența de tăiere 3,3*kHz*). Ieșirea filtrului de antialiere este conectată la intrarea analogică *AN0* controlerului. Dacă la intrarea analogică este plasat un microfon și se face selecția pentru acesta, semnalului provenind de la acesta i se adaugă o tensiune continuă de 1,65*V*. În modul de intrare *Line*, nu este necesară această tensiune.

Amplificatorul are o creștere care variază de la 3*dB* la 23*dB*, aceasta putând fi ajustată pentru a controla sensibilitatea microfonului sau creșterea semnalului slab de la linia de intrare.

Dacă nu este folosită intrarea analogică din microcontroler, poate fi utilizat codecul audio care asigură o calitate mai bună a semnalului în domeniul audio. Intrarea codecului audio este conectată direct la ieșirea preamplificatorului. Codecul comunică cu microcontrolerul prin intermediul a două canale de comunicație. Prin protocolul I^2C codecul primește, respectiv transmite, comenzile sau stările în care se află (de exemplu: protocolul de comunicație, rata de eșantionare, controlul volumului, controlul nivelului, setările filtrului, etc.), iar prin interfața *DCI* (*Data Converter Interface*) transmite și primește secvențele de date

care trebuie să le transpună în mediul analogic. Pentru operarea cu interfața DCI este implementat un protocol de comunicație specific, care impune transmisii de cadre, având dimensiunea maximă de 256 biți și suportând până la 16 intervale de transmisie separate (*time slots*). În vederea comunicării cu mediul analogic, sistemul mai dispune de o linie digitală pe care se poate transmite semnalului sub formă de impulsuri modulate în durată (PWM – *Pulse Width Modulation*). Semnalul este apoi filtrat cu un filtru trece jos de ordinul întâi, după cum se vede în figura de mai jos:



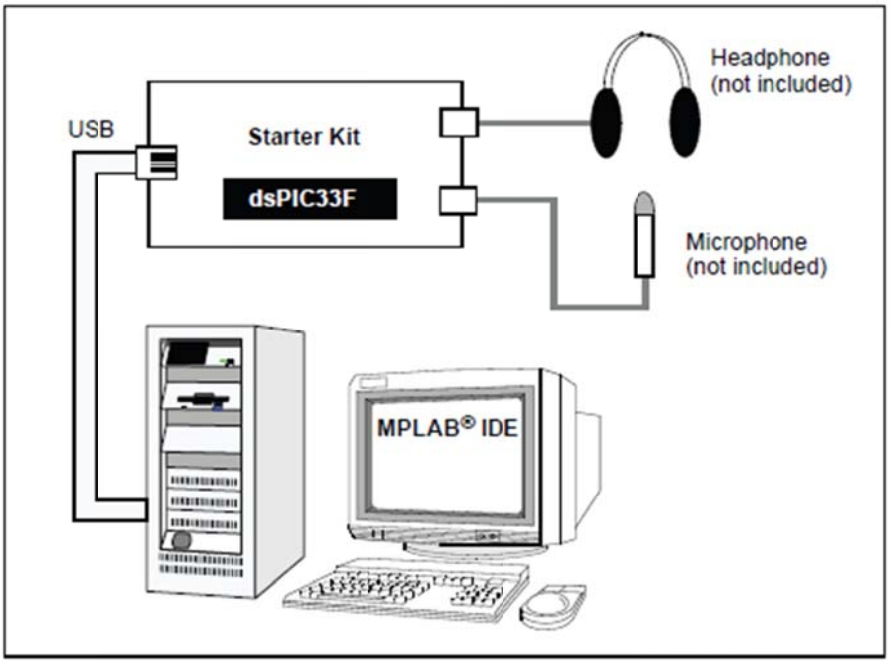
Filtrul se comportă ca un integrator, iar frecvența semnalului PWM trebuie să fie un multiplu întreg al ratei de eșantionare.

Dispozitivul *Starter Kit* include și o memorie serială de tip flash, de 4*MB*, ce poate fi utilizată pentru stocare de informații. Memoria este conectată la magistrala *SPI* de pe dispozitivul dsPIC33F și poate fi utilizată, de obicei, de aplicațiile care necesită depozitarea eșantioanelor semnalului audio înregistrat pentru a putea fi redat.

Sistemul mai deține un modul integrat de programare a microcontrolerului, dar permite și lucrul în modul *Debug*. Programele pot fi scrise în limbajul C, dar și în limbaj de asamblare, în mediul de programarea MPLAB. Compilarea programului C se face cu pachetul software C30. În cadrul mediului de dezvoltare MPLAB se găsesc o serie de biblioteci implementate direct in limbaj de asamblare ce pot fi apelate în limbajul C, pentru a fi utilizate în algoritmi de procesarea digitală a semnalelor.

Microsistemul este conectat la mediul exterior prin intermediul a două mufe jack 3,5mm, pentru semnalul analogic de intrare, respectiv de ieșire, un port USB, prin care face transferul datelor digitale cu mediul MPLAB, două butoane și două LED-uri de uz general.

Schemă pentru realizarea aplicației:



Această figură conține:

- placa de dezvoltare *DSC Starter Kit* cu microcontroler dsPIC33F;
- microfonul și căștile care trebuie conectate la placă;
- conexiunea USB ce asigură comunicarea și alimentarea plăcii de dezvoltare.

Aplicația software pentru dispozitivul dsPIC33F redă mesajul salvat în memoria flash de pe placă și permite înregistrarea și redarea mesajului înregistrat.

3.1. Conectarea plăcii

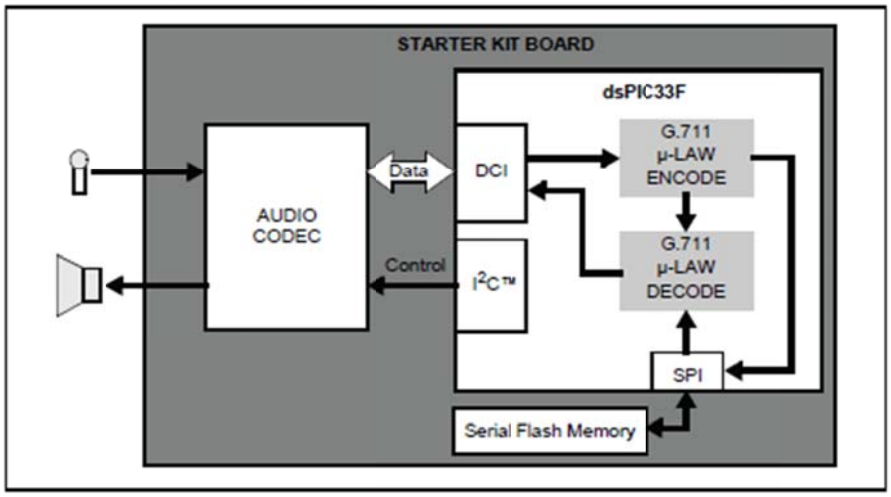
Pentru conectarea plăcii trebuie urmăriți pașii:

- conectarea microfonului la ieșirea J9;
- conectarea căștilor la ieșirea J8;
- alimentarea plăcii prin conectarea acesteia la calculator prin portul USB;
- când placa va fi alimentată aplicația va reda un mesaj introductiv.

3.2. Redarea aplicației

Pentru redarea aplicației trebuie să urmărim pașii:

- pentru a înregistra un mesaj se apasă butonul S1 și se așteaptă până când LED-ul roșu se stinge (memoria flash serială a fost ștearsă) și LED-ul galben se aprinde; aplicația va înregistra semnalul primit de la microfon și îl va salva în memoria flash;
- se apasă butonul S2 pentru redarea mesajului salvat; LED-ul verde se aprinde cât timp mesajul este redat;
- apăsând butonul S1 din nou, memoria flash va fi ștearsă și se pregătește sistemul pentru o altă înregistrare.



4. Desfășurarea laboratorului

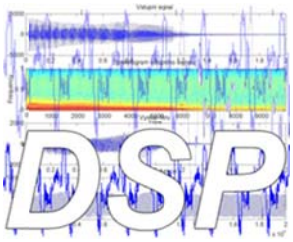
În continuare sunt prezentate câteva exemple de programe care pun în evidență:

- eșantionarea și redarea imediată a unei secvențe (realizarea ecoului);
- un generator de semnal sinusoidal;
- un generator haotic pe baza ecuației logistice.

1. Eșantionarea și redarea imediată a unei secvențe

Acest proiect ilustrează preluarea și redarea imediată a unei secvențe analogice de semnal. Pentru această aplicație sunt necesare mai multe fișiere, care pot fi preluate din programele exemplu din *C:\Microchip Starter Kits\dsPIC Starter Kit - 1\demo*. Așadar, din *C:\Microchip Starter Kits\dsPIC Starter Kit - 1\demo\SASK WM8510 Record Play Demo With Intro* se adaugă fișierele:





- *WM8510CodecDrv.h*, din directorul *h*;
- *WM8510CodecDrv.c*, din directorul *src*.

Fișierul antet conține declarațiile funcțiilor și constantelor care permit utilizarea codecului de pe placă, iar fișierul sursă conține codul C/C++ aferent fiecărei funcții.

Se va deschide un fișier nou, sa va salva cu denumirea *main*, și se va adăuga proiectului.

Programul principal folosit:

```
#include "..\h\p33FJ256GP506.h"
#include "..\h\WM8510CodecDrv.h"
//#include "..\h\ADCChannelDrv.h" //se adauga proiectului si
ADCChannelDrv.h si ADCChannelDrv.c

_FGS(GWRP_OFF & GCP_OFF); //Fara protectie la scriere si fara
protectia codului
_FOSCSEL(FNOSC_FRC); //Se foloseste oscilatorul RC rapid
(Fast RC)
_FOSC(FCKSM_CSECMD & OSCIOFNC_ON & POSCMD_NONE); //Se
activeaza comutarea ceasurilor, OSC2 ca pin digital, se
dezactiveaza oscilatorul primar
_FWDT(FWDTEN_OFF); //Se opreste cronometrul watchdog

#define FRAME_SIZE    128
int codecBuffer    [WM8510DRV_DRV_BUFFER_SIZE];

//Instantierea driverelor
//ADCChannelHandle adcChannelHandle;
//ADCChannelHandle *pADCChannelHandle = &adcChannelHandle;

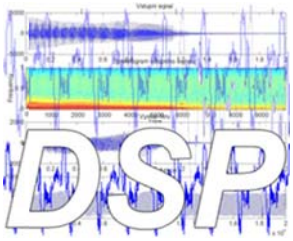
WM8510Handle codec;
WM8510Handle * codecHandle = &codec;


//Programul principal
int main (){

    int samples [FRAME_SIZE];

    //Configurarea oscilatorului astfel incat dispozitivul sa
opereze la 40MHz
    //Fosc= Fin*M/(N1*N2), Fcy=Fosc/2
    //Fosc= 7.37M*40/(2*2)=80Mhz pentru tact intrare de 7.37M
    PLLFBD=41;                  //M=39
    CLKDIVbits.PLLPOST=0;       //N1=2
    CLKDIVbits.PLLPRE=0;        //N2=2
    OSCTUN=0;
```






```
    __builtin_write_OSCCONH(0x01); //Initializare oscilator
la FRC (Fast RC) cu PLL
    __builtin_write_OSCCONL(0x01);
    while (OSCCONbits.COSC != 0b01); //Astepta realizarea
comutarii
    while(!OSCCONbits.LOCK);

    // Initializari
    WM8510Init(codecHandle,codecBuffer); //Initializeaza
codecul
    WM8510Start(codecHandle); //Porneste codecul
    WM8510SampleRate8KConfig(codecHandle); //Configureaza
codecul sa functioneze cu frecventa de esantionare de 8kHz

    //Bucla de program
    while(1){
        //Esantionarea semnalului de la microfon
        //Asteapta ca WM8510 sa devina disponibil
        while(WM8510IsReadBusy(codecHandle));
        //Esantioneaza semnalul audio
        WM8510Read(codecHandle,samples,FRAME_SIZE);

        //Asteapta ca WM8510 sa devina disponibil
        while(WM8510IsWriteBusy(codecHandle));
        //Redare semnal inregistrat
        WM8510Write(codecHandle,samples,FRAME_SIZE);
    }
}
```

Pentru a nu avea probleme la compilare, se va adăuga fișierul *WM8510CodecDrv.h* în arborele de căutare a fișierelor antet: *Project → Build Options… → Project → Directories → Show directories for: Include Search Path → New → … → C:\Microchip Starter Kits\dsPIC Starter Kit - 1\demo\SASK WM8510 Record Play Demo With Intro\h* și se apasă *OK*.

Se observă ca eșantioanele preluate de la codec sunt salvate în vectorul tampon *samples*, după care sunt redate.

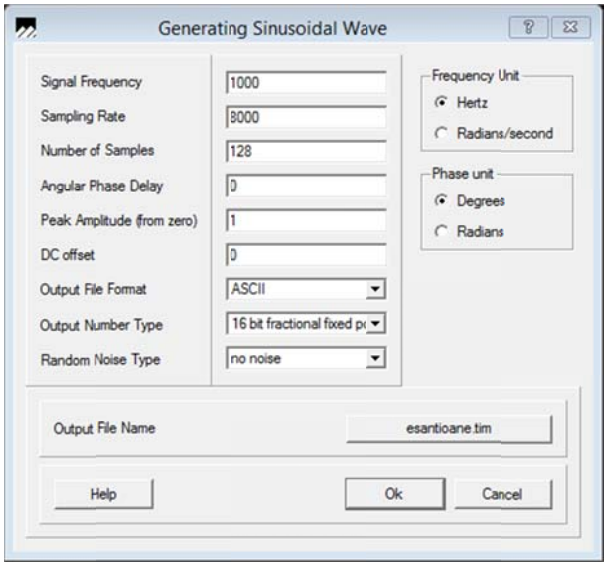
Acest program poate fi modificat pentru a folosi convertorul analog-numeric integrat în microcontroler, pentru eșantionarea și conversia semnalului analogic de intrare. Astfel, se vor folosi liniile comentate din programul de mai sus, și funcțiile:

- `void ADCChannelInit(ADCChannelHandle * pHandle, int * pBufferInDMA);`
- `void ADCChannelStart(ADCChannelHandle * pHandle);`
- `void ADCChannelRead(ADCChannelHandle * pHandle, int *buffer,int size);`
- `int ADCChannelIsBusy(ADCChannelHandle * pHandle).`

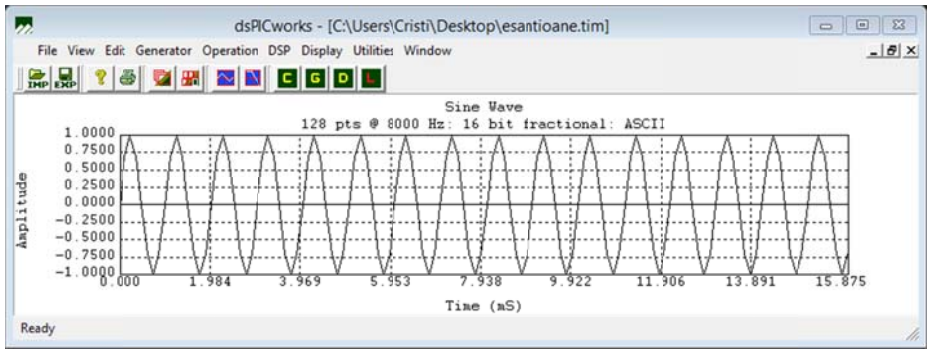
Observație. Trebuie aleasă ieșirea de codec pe placă, nu de OCWPM. Așadar, J6 trebuie să fie pe poziția CODEC.

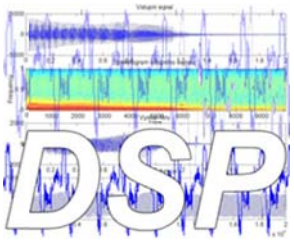
2. Generator de semnal sinusoidal

Se va genera un semnal sinusoidal de frecvență fixă, 1*kHz*. Pentru a realiza această aplicație se vor folosi eșantioanele unui semnal sinusoidal de frecvență 1*kHz*, obținute cu ajutorul programului *dsPICworks v2.0*. Acestea vor fi salvate într-un vector care va fi citit în mod repetat. Pentru generarea eșantioanelor se deschide programul *dsPICworks v2.0*. Se selectează *Generator* → *Sinusoidal…*, și se introduc datele din imagine:



Forma de undă generată:





Fișierul *esantioane.tim* se deschide cu aplicația *Notepad*, iar valorile se vor copia în programul principal.

Se vor adăuga proiectului fișierele *WM8510CodecDrv.h* și *WM8510CodecDrv.c*, care conțin declarațiile și codul funcțiilor necesare pentru generarea semnalului audio folosind codecul. De asemenea, se va adăuga arborelui de căutare locația *C:\Microchip Starter Kits\dsPIC Starter Kit - 1\demo\SASK WM8510 Record Play Demo With Intro\h*.

Programul principal folosit:

```
#include "..\h\p33FJ256GP506.h"
#include "..\h\WM8510CodecDrv.h"

_FGS(GWRP_OFF & GCP_OFF); //Fara protectie la scriere si fara
protectia codului
_FOSCSEL(FNOSC_FRC); //Se foloseste oscilatorul RC rapid
(Fast RC)
_FOSC(FCKSM_CSECMD & OSCIOFNC_ON & POSCMD_NONE); //Se
activeaza comutarea ceasurilor, OSC2 ca pin digital, se
dezactiveaza oscilatorul primar
_FWDT(FWDTEN_OFF); //Se opreste cronometrul watchdog

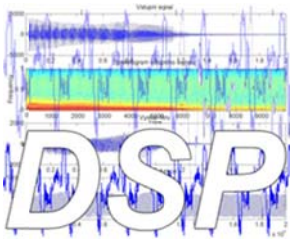
#define FRAME_SIZE 128
int codecBuffer    [WM8510DRV_DRV_BUFFER_SIZE];

//Esantioanele unui semnal de 1kHz esantionat cu frecventa de
8kHz
int samples[FRAME_SIZE] =
{0x0000,0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,0x8001,
0xA57F,0x0000,0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,
0x0000,0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,
0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,0x5A81,
0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,0x5A81,0x7FFF,
0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,0x5A81,0x7FFF,0x5A81,
0x0000,0xA57F,0x8001,0xA57F,0x0000,0x5A81,0x7FFF,0x5A81,0x0000,
0xA57F,0x8001,0xA57F,0x0000,0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,
0x8001,0xA57F,0x0000,0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,0x8001,
0xA57F,0x0000,0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,
0x0000,0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,
0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,0x5A81,
0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,0x5A81,0x7FFF,
0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,0x5A81,0x7FFF,0x5A81,
0x0000,0xA57F,0x8001,0xA57F};

//Instantierea driverului
WM8510Handle codec;
WM8510Handle * codecHandle = &codec;

//Programul principal
int main (){
```






```
        //Configurarea oscilatorului astfel incat
dispozitivul sa opereze la 40MHz
        //Fosc= Fin*M/(N1*N2), Fcy=Fosc/2
        //Fosc= 7.37M*40/(2*2)=80Mhz pentru tact intrare de
7.37M
        PLLFBD=41;                      //M=39
        CLKDIVbits.PLLPOST=0;           //N1=2
        CLKDIVbits.PLLPRE=0;            //N2=2
        OSCTUN=0;

        __builtin_write_OSCCONH(0x01); //Initializare
oscilator la FRC (Fast RC) cu PLL
        __builtin_write_OSCCONL(0x01);
        while (OSCCONbits.COSC != 0b01);//Astepta realizarea
comutarii
        while(!OSCCONbits.LOCK);

        // Initializari
        WM8510Init(codecHandle,codecBuffer); //Initializeaza
codecul
        WM8510Start(codecHandle); //Porneste codecul

        //Bucla de program
        while(1){
                //Citirea esantioanelor semnalului
                //Asteapta ca WM8510 sa devina disponibil
                while(WM8510IsWriteBusy(codecHandle));
                //Redare semnal inregistrat
                WM8510Write(codecHandle,samples,FRAME_SIZE);
        }
}
```

3. Generator haotic pe baza ecuației logistice

Există multe aplicații în care este necesară generarea numerelor aleatoare. Una dintre ele o reprezintă generarea unui semnal de zgomot.

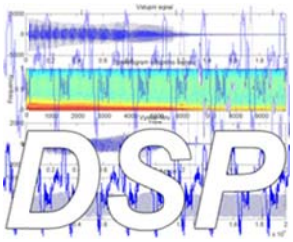
Un exemplu tipic de generator haotic în timp discret este cel descris de ecuația logistică:

$$x_n = r \cdot x_{n-1}\left(1 - x_{n-1}\right)$$

Acesta este un proces iterativ prin care eșantionul curent este calculat în funcție de eșantionul precedent. Pentru $r = 0,7$, relația devine:

$$x_n = 0,7 \cdot x_{n-1}\left(1 - x_{n-1}\right)$$





Programul principal folosit:

```
#include "..\h\p33FJ256GP506.h"

_FGS(GWRP_OFF & GCP_OFF); //Fara protectie la scriere si fara
protectia codului
_FOSCSEL(FNOSC_FRC); //Se foloseste oscilatorul RC rapid
(Fast RC)
_FOSC(FCKSM_CSECMD & OSCIOFNC_ON & POSCMD_NONE); //Se
activeaza comutarea ceasurilor, OSC2 ca pin digital, se
dezactiveaza oscilatorul primar
_FWDT(FWDTEN_OFF); //Se opreste cronometrul watchdog

#define FRAME_SIZE 128

//Programul principal
int main (){

        //Configurarea oscilatorului astfel incat
dispozitivul sa opereze la 40MHz
        //Fosc= Fin*M/(N1*N2), Fcy=Fosc/2
        //Fosc= 7.37M*40/(2*2)=80Mhz pentru tact intrare de
7.37M
        PLLFBD=41;                      //M=39
        CLKDIVbits.PLLPOST=0;           //N1=2
        CLKDIVbits.PLLPRE=0;            //N2=2
        OSCTUN=0;

        __builtin_write_OSCCONH(0x01); //Initializare
oscilator la FRC (Fast RC) cu PLL
        __builtin_write_OSCCONL(0x01);
        while (OSCCONbits.COSC != 0b01);//Astepta realizarea
comutarii
        while(!OSCCONbits.LOCK);

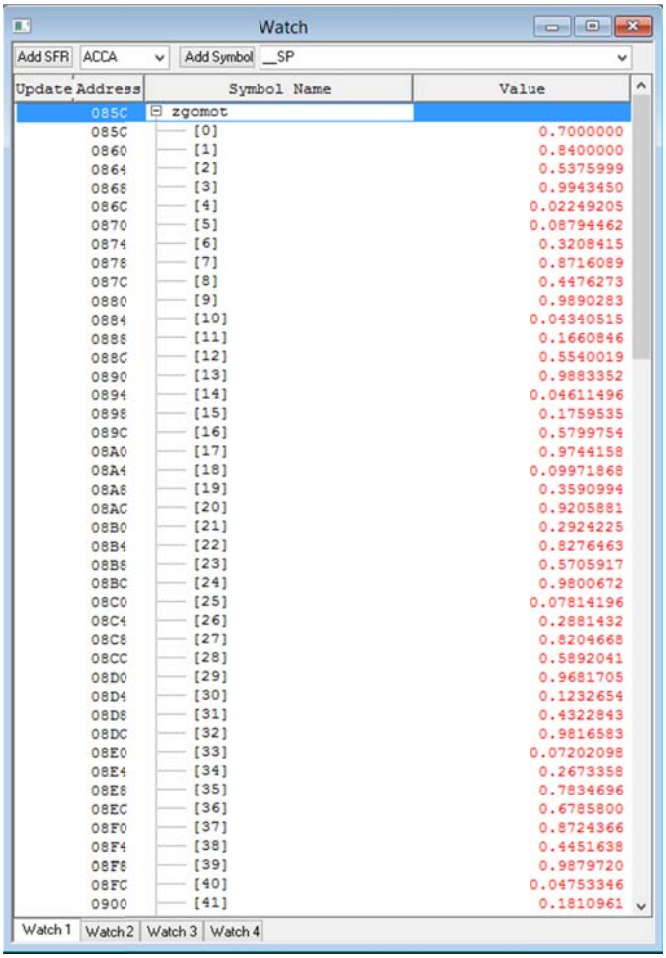
        //Generare numere aleatoare folosind ecuatia
logisticii
        float zgomot[FRAME_SIZE];
        zgomot[0] = 0.7;
        int j = 1;
        int r = 4;

        while (j<=FRAME_SIZE) {
                zgomot[j] = r*zgomot[j-1]*(1-zgomot[j-1]);
                j++;
        }

        //Bucla de program
        while(1){
```

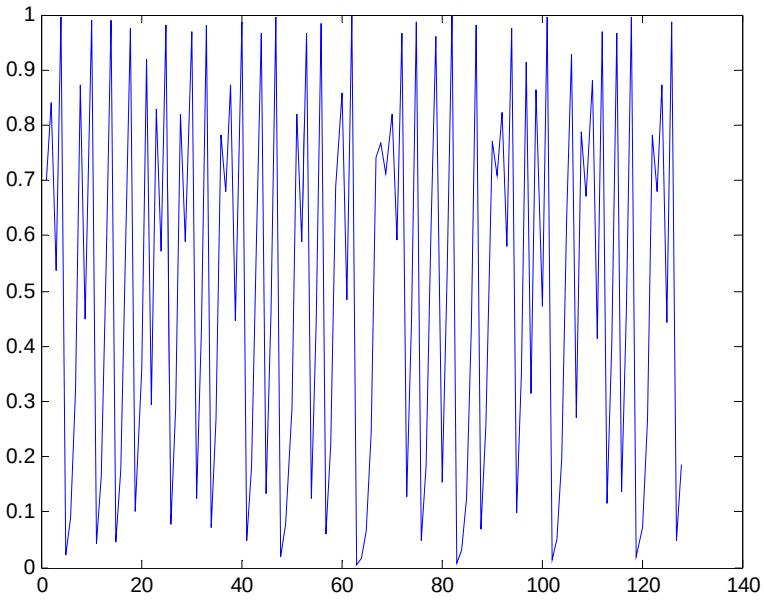
```
        //Instructiuni...
        }
}
```

Pentru a vizualiza semnalul generat, se poate observa conținutul vectorului *zgomot*. Pentru aceasta se selectează *Debugger → Select Tool → 11 Starter Kits*. Se alege *Project → Build Configuration → Debug*. După compilare, se adaugă vectorul *zgomot* în fereastra de observație: *View → Watch → Selectați zgomot → Add Symbol*. Se programează microcontrolerul şi se rulează programul. După câteva secunde se poate opri execuția programului pentru a vizualiza conținutul vectorului *zgomot*:



Se execută click dreapta pe *zgomot*, se alege *Export Table…*, se apasă *OK*, se alege *Single Column Output* și se alege numele fișierului în care se va salva conținutul vectorului. Acesta se va deschide cu aplicația *Notepad*, se va copia conținutul său și se va realiza un grafic folosind instrucțiunea *plot([*vector_date*])*, din MATLAB.

Semnalul generat:



Realizați următoarele aplicații:

a) Realizați un program prin care să se dubleze amplitudinea unui semnal de intrare. Semnalul de ieșire se va genera folosind codul audio.

b) Modificați programul prin care procesorul de semnal generează un semnal sinusoidal astfel încât semnalul de ieșire să aibă frecvența de 30*Hz*.

c) Modificați același program astfel încât semnalul de ieșire să fie de 500*Hz* și frecvența de eșantionare de 48*kHz*.

d) Generați un semnal format din două sinusoide cu frecvențele de 100*Hz*, respectiv 300*Hz*, iar frecvența de eșantionare fiind de 48*kHz*.

e) Realizați un program astfel încât semnalul de ieșire să respecte următoarea relație:

$$y[n] = x[x] + x[n-1] + x[n-2]$$

f) Realizați un program prin care să generați un semnal triunghiular, cu frecvența de 500*Hz* și frecvența de eșantionare de 48*kHz*.